UPTS Testing Guide

How to install the tools and run every test in the project, step by step, on Windows and Linux.

SE3110 Quality Management in Software Engineering · SE3090 integrated system (UPTSLK)

Part	What you will run	Tool
Backend	API unit, integration, security and database tests	xUnit (dotnet test)
Web	React component, form, route and API tests	Vitest (pnpm test)
Mobile	Flutter unit, widget and API tests	flutter test
Performance	Load tests, one script per member	k6
Security	Automated scan of the whole API	OWASP ZAP

Member	Component	Backend folder	Web folder	k6 script
Kishan (kishan-ahamed45)	A - Centres and Network	`Centres/`	`src/test/centres/`	`centres-network.js`
Rashmi (RashmiK0119)	B - Fleet and Maintenance	`Fleet/`	`src/test/fleet/`	`fleet-maintenance.js`
Chamal (chamals3n4)	C - Scheduling and Dispatch	`Dispatch/`	`src/test/dispatch/`	`dispatch-trips.js`
Nadeesha (Nadeesha-D-Shalom)	D - Passengers and Fares	`Bookings/`	`src/test/bookings/`	`bookings-wallet.js`

Shared by the whole team: backend `Auth/`, `AgentRecovery/`, `Database/`, `Integration/`, `Notifications/`, all mobile tests, and the OWASP ZAP scan.

1. What you need

You do not need every tool. Install only what you will run. The table says which part needs which tool.

Tool	Version	Needed for	Check it works
Git	any	getting the code	`git --version`
.NET SDK	8.0	backend tests, running the API	`dotnet --version`
Node.js	18 or newer (22 recommended)	web tests	`node -v`
pnpm	9 or newer	web tests	`pnpm -v`
Flutter SDK	latest stable (Dart 3.13+)	mobile tests	`flutter --version`
Docker	Docker Desktop or Docker Engine	backend Database tests, local PostgreSQL for k6 and ZAP	`docker --version`
k6	latest	performance tests	`k6 version`
Java	17 or newer	OWASP ZAP	`java -version`
OWASP ZAP	2.16 or newer	security scan	`see section 7`

Tip: after installing anything, close and reopen your terminal so the new PATH is picked up. Then run the commands in the last column. Each should print a version number.

1.1 Install on Windows (PowerShell)

Open **PowerShell** (Start menu → type PowerShell). `winget` is built into Windows 10 and 11.

Everything except Flutter

```
winget install --id Git.Git -e
winget install --id Microsoft.DotNet.SDK.8 -e
winget install --id OpenJS.NodeJS.LTS -e
npm install -g pnpm
winget install --id Docker.DockerDesktop -e
winget install k6 --source winget
winget install --id Microsoft.OpenJDK.21 -e
winget install --id ZAP.ZAP -e
```

Flutter on Windows: download the Windows zip from `https://docs.flutter.dev/get-started/install/windows`, extract it to `C:\src\flutter`, then add `C:\src\flutter\bin` to PATH (Start → *Edit the system environment variables* → Environment Variables → Path → New). Then:

```
flutter --version
flutter doctor
```

Start **Docker Desktop** once from the Start menu and wait until it says it is running before you use Docker.

1.2 Install on Linux (Ubuntu / Debian)

Tools from the package manager

```
sudo apt-get update
sudo apt-get install -y git curl dotnet-sdk-8.0 openjdk-17-jre docker.io docker-compose-v2

# Node.js 22 and pnpm
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt-get install -y nodejs
sudo npm install -g pnpm

# Flutter and OWASP ZAP (snap)
sudo snap install flutter --classic
sudo snap install zaproxy --classic

# Let your user run Docker without sudo (log out and in afterwards)
sudo usermod -aG docker $USER
```

k6 (official repository)

```
sudo gpg -k
sudo gpg --no-default-keyring --keyring /usr/share/keyrings/k6-archive-keyring.gpg \
  --keyserver hkp://keyserver.ubuntu.com:80 --recv-keys \
    C5AD17C747E3415A3642D57D77C6C491D6AC1D69
echo "deb [signed-by=/usr/share/keyrings/k6-archive-keyring.gpg] \
https://dl.k6.io/deb stable main" | sudo tee /etc/apt/sources.list.d/k6.list
sudo apt-get update && sudo apt-get install -y k6
```

1.3 Get the code

```
git clone https://github.com/sliit-y3s1-projects/uptslk.git
cd uptslk
```

All commands in this guide start from this `uptslk` folder (the repository root) unless a `cd` is shown. The commands are the same on Windows and Linux unless a separate Windows line is given.

2. Backend tests (xUnit)

The backend tests start the real API inside the test process (`WebApplicationFactory`) with an in-memory SQLite database, so **they need only the .NET SDK**. The one exception is the `Database/` folder, which starts a real PostgreSQL in Docker (Testcontainers), so Docker must be running for those.

Run everything that does not need Docker (recommended first run)

```
cd apps/backend
dotnet test api.Tests/api.Tests.csproj --filter "Category!=Database"
```

With Docker running

```
# everything (Docker must be running)
dotnet test api.Tests/api.Tests.csproj
# only the PostgreSQL tests
dotnet test api.Tests/api.Tests.csproj --filter "Category=Database"
```

The first run downloads packages and builds, so it takes a minute or two. A good result ends with `Passed! - Failed: 0`.

2.1 Each member's backend tests

Member	Command (run inside apps/backend)
Kishan - Centres	`dotnet test api.Tests/api.Tests.csproj --filter "FullyQualifiedName~api.Tests.Centres."`
Rashmi - Fleet	`dotnet test api.Tests/api.Tests.csproj --filter "FullyQualifiedName~api.Tests.Fleet."`
Chamal - Dispatch	`dotnet test api.Tests/api.Tests.csproj --filter "FullyQualifiedName~api.Tests.Dispatch."`
Nadeesha - Bookings	`dotnet test api.Tests/api.Tests.csproj --filter "FullyQualifiedName~api.Tests.Bookings."`
Shared - Auth and security	`dotnet test api.Tests/api.Tests.csproj --filter "FullyQualifiedName~api.Tests.Auth."`
Shared - AI agent recovery	`dotnet test api.Tests/api.Tests.csproj --filter "FullyQualifiedName~api.Tests.AgentRecovery."`
Shared - End-to-end journey	`dotnet test api.Tests/api.Tests.csproj --filter "FullyQualifiedName~api.Tests.Integration."`
Shared - Notifications	`dotnet test api.Tests/api.Tests.csproj --filter "FullyQualifiedName~api.Tests.Notifications."`

> The dot at the end of the folder name matters: `api.Tests.Fleet.` matches only the Fleet folder.

2.2 One test class or one single test

```
dotnet test api.Tests/api.Tests.csproj --filter "FullyQualifiedName~TripConflictTests"
dotnet test api.Tests/api.Tests.csproj --filter \
    "Name=SameVehicleAtOverlappingTime_IsRejected"
```

Find class names in the file names (for example `Dispatch/TripConflictTests.cs`) and test names in the methods marked `[Fact]` or `[Theory]`.

2.3 See every test name, save results, measure coverage

```
# list each test and its result
dotnet test api.Tests/api.Tests.csproj --filter \
    "FullyQualifiedName~api.Tests.Dispatch." --logger "console;verbosity=detailed"

# save results to a file (api.Tests/TestResults/results.trx)
dotnet test api.Tests/api.Tests.csproj --filter "Category!=Database" --logger \
    "trx;LogFileName=results.trx"

# code coverage, then an HTML report
dotnet test api.Tests/api.Tests.csproj --filter "Category!=Database" --settings \
    api.Tests/coverage.runsettings --results-directory api.Tests/TestResults/cov
dotnet tool install -g dotnet-reportgenerator-globaltool
reportgenerator -reports:"api.Tests/TestResults/cov/**/coverage.cobertura.xml" \
    -targetdir:coverage-html -reporttypes:Html
```

Open `apps/backend/coverage-html/index.html` in a browser to see coverage per class.

3. Web frontend tests (Vitest)

The web tests render the React pages in a simulated browser (jsdom) and answer every API call with a mock (MSW), so **no API or database is needed**.

Run everything

```
cd apps/web
pnpm install          # first time only
pnpm test             # all web tests
```

3.1 Each member's web tests

Member	Command (run inside apps/web)
Kishan - Centres	`pnpm test src/test/centres`
Rashmi - Fleet	`pnpm test src/test/fleet`
Chamal - Dispatch	`pnpm test src/test/dispatch`
Nadeesha - Bookings	`pnpm test src/test/bookings`
Shared - test set-up checks	`pnpm test src/test/foundation.test.tsx`

3.2 One file, or tests whose name matches

```
pnpm test src/test/dispatch/TripForm.test.tsx
pnpm test -t "requires a reason"
```

3.3 Watch mode, coverage and lint

```
pnpm test:watch       # re-runs tests when you save a file (press q to quit)
pnpm test:coverage    # report in apps/web/coverage/index.html
pnpm lint             # ESLint checks
pnpm build            # TypeScript check + production build
```

4. Mobile tests (Flutter)

The Flutter tests run on your computer with `flutter test`. No emulator, phone or API is needed: API calls are answered by a fake HTTP client.

Run everything

```
cd apps/mobile
flutter pub get       # first time only
flutter test          # all mobile tests
```

4.1 One area or one file

What	Command (run inside apps/mobile)
Sign-in and register forms, navigation	`flutter test test/features/auth_forms_test.dart`
Driver incident report form	`flutter test test/features/report_incident_test.dart`
Sign-in session (AuthStore)	`flutter test test/state/auth_store_test.dart`
Notifications state	`flutter test test/state/notification_store_test.dart`
All API service tests	`flutter test test/services`
Trip search API	`flutter test test/services/trip_search_api_service_test.dart`
Booking, driver, auth, notification APIs	`flutter test test/services/<name>_api_service_test.dart`
Booking eligibility rules	`flutter test test/models/booking_eligibility_test.dart`
Journey search screen sizes	`flutter test test/journey_search_widget_test.dart`
Tests whose name matches	`flutter test --name "rejects bad input"`

```
flutter test --coverage      # writes coverage/lcov.info
flutter analyze              # static checks
```

5. Start a local API (for k6 and ZAP only)

Performance and security tests send real requests to a running API. They **create and change data**, so always use a separate, empty database, never the shared or deployed one. Sections 2 to 4 do not need this.

Step 1 - start PostgreSQL and create an empty test database (repository root)

```
docker compose up -d
docker compose exec postgres createdb -U upts_user upts_perf
```

Step 2 - start the API on http://localhost:5250 (Linux)

```
cd apps/backend/api
DB="Host=localhost;Port=5433;Database=upts_perf;"
DB+="Username=upts_user;Password=upts_password"
ConnectionStrings__DefaultConnection="$DB" dotnet run -c Release --launch-profile http
```

Step 2 - Windows PowerShell

```
cd apps/backend/api
$db = "Host=localhost;Port=5433;Database=upts_perf;" + `
    "Username=upts_user;Password=upts_password"
$env:ConnectionStrings__DefaultConnection = $db
dotnet run -c Release --launch-profile http
```

The API uses your usual local settings (JWT keys in `appsettings.Development.json` or user secrets, the same as when you run the app). On an empty database it creates all tables and the bootstrap admin (`admin@upts.lk`, password from your `BootstrapAdmin` settings; the fallback is `admin123`). Leave this terminal open and check in another one:

```
curl http://localhost:5250/health        # prints Healthy
```

6. Performance tests (k6)

Each script builds its own centre, buses, drivers, trips and passengers through the API, then runs its scenarios with 10 virtual users for 30 seconds each. A script **passes** when every threshold is met: response time (p95) under the limit, fewer than 1% failed requests, and more than 99% passing checks. k6 prints a tick or a cross per threshold.

Member	Linux / macOS (repository root)
Kishan	`K6_EMAIL=admin@upts.lk K6_PASSWORD=admin123 k6 run perf/k6/centres-network.js`
Rashmi	`K6_EMAIL=admin@upts.lk K6_PASSWORD=admin123 k6 run perf/k6/fleet-maintenance.js`
Chamal	`K6_EMAIL=admin@upts.lk K6_PASSWORD=admin123 k6 run perf/k6/dispatch-trips.js`
Nadeesha	`K6_EMAIL=admin@upts.lk K6_PASSWORD=admin123 k6 run perf/k6/bookings-wallet.js`

Windows PowerShell (set the login once, then run any script)

```
$env:K6_EMAIL = "admin@upts.lk"
$env:K6_PASSWORD = "admin123"
k6 run perf/k6/dispatch-trips.js
```

Setting	Default	Meaning
`K6_BASE_URL`	`http://localhost:5250`	where the API runs
`K6_EMAIL / K6_PASSWORD`	`admin@upts.lk / admin123`	admin login used by the script
`PERF_VUS`	`10`	virtual users (simulated people at the same time)

Setting	Default	Meaning
`PERF_DURATION`	`30s`	length of each scenario, in seconds
`PERF_TOPUPS`	`200`	wallet top-ups in bookings-wallet.js

Save the result as evidence

```
k6 run --quiet --no-color --summary-export \
    docs/testing/evidence/k6-dispatch-trips-summary.json \
  perf/k6/dispatch-trips.js > docs/testing/evidence/k6-dispatch-trips.log 2>&1
```

The older shared scripts still work too: `perf/k6/read-apis.js` and `perf/k6/booking-create.js`. Full details are in `perf/k6/README.md`.

7. Security scan (OWASP ZAP)

ZAP reads every endpoint from the API's Swagger document, signs in as the admin and runs a passive and an active (attack) scan. It needs the local API from section 5. The plan is `security/zap/api-scan.yaml`.

Linux (repository root)

```
export ZAP_TARGET=http://localhost:5250
export ZAP_REPORTS="$PWD/docs/testing/evidence/zap"
LOGIN='{"email":"admin@upts.lk","password":"admin123"}'
export ZAP_TOKEN=$(curl -s -X POST "$ZAP_TARGET/api/v1/auth/login" \
    -H "Content-Type: application/json" -d "$LOGIN" \
    | python3 -c 'import sys,json; print(json.load(sys.stdin)["token"])')
# snap install; with the ZAP download use /path/to/ZAP/zap.sh instead of zaproxy
zaproxy -cmd -autorun "$PWD/security/zap/api-scan.yaml"
```

Windows PowerShell (repository root)

```
$env:ZAP_TARGET  = "http://localhost:5250"
$env:ZAP_REPORTS = "$PWD\docs\testing\evidence\zap"
$login = '{"email":"admin@upts.lk","password":"admin123"}'
$env:ZAP_TOKEN = (Invoke-RestMethod -Method Post -Uri `
    "$env:ZAP_TARGET/api/v1/auth/login" `
    -ContentType "application/json" -Body $login).token
& "C:\Program Files\ZAP\Zed Attack Proxy\zap.bat" -cmd -autorun `
    "$PWD\security\zap\api-scan.yaml"
```

The scan takes about 3-5 minutes. Open `docs/testing/evidence/zap/zap-api-scan.html` for the report. The plan must be given with its **full path** as shown, otherwise ZAP looks for it in its own folder. Findings and their status are listed in `security/zap/README.md`.

8. Showing a test in the viva

A short demonstration that works for any member:

Step	What to do	What it shows
1	Run your own folder (section 2.1 or 3.1) and point at `Passed! - Failed: 0`.	your tests run and pass
2	Open one of your test files and explain one test: the set-up (Arrange), the request or action (Act) and the checks (Assert).	you understand the test
3	Change one expected value, e.g. `HttpStatusCode.Conflict` to `HttpStatusCode.OK`, run that single test (section 2.2) and show it fail with a clear message.	the test really checks something
4	Undo the change (`git checkout -- <file>`) and run it again: green.	you can modify and re-run
5	Show a defect you found: the before-fix log, the fix commit and the after-fix log in `docs/testing/evidence/`.	defects, fixes and retesting

9. Troubleshooting

Problem	Fix
`dotnet`, `pnpm`, `flutter` or `k6`: command not found	Close and reopen the terminal. If it still fails, the tool's folder is not on PATH; reinstall it or add it to PATH.
Database tests fail with a Docker error	Start Docker Desktop (Windows) or `sudo systemctl start docker` (Linux), or skip them with `--filter "Category!=Database"`.
Port 5433 already in use	Another PostgreSQL is running. Stop it, or change the port in `docker-compose.yml` and in the connection string.
k6 setup fails with "Login ... failed: HTTP 401"	The admin email or password is wrong. Use the values from your BootstrapAdmin settings.
k6 or ZAP: connection refused	The API is not running. Start it (section 5) and check `http://localhost:5250/health`.
`pnpm test` says Vitest is missing	Run `pnpm install` inside `apps/web` first.
`flutter test` asks for packages	Run `flutter pub get` inside `apps/mobile` first.
ZAP: "Cannot access file"	Give the plan with its full path, as in section 7.
PowerShell refuses to run scripts	Run once: `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`.